



# Spack Tutorial - CINECA 2026

## **EB + EESSI + Spack Hackathon**

CINECA (BO), May 19-21, 2026

---

Harmen Stoppels, Massimiliano Culpo

# Who are we?



[github.com/alalazo](https://github.com/alalazo)



[github.com/haampie](https://github.com/haampie)



[slack.spack.io](https://slack.spack.io)

# Introduction to Spack

# What is Spack?

- Spack automates the build and installation of scientific software
- Packages are *parameterized*, so that users can easily tweak and tune configuration

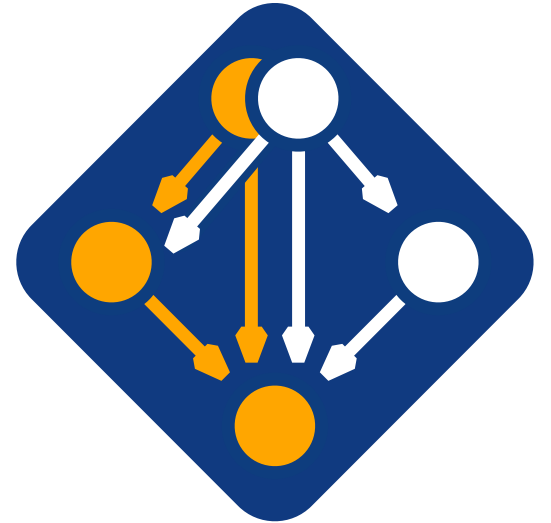
## No installation required: clone and go

```
$ git clone https://github.com/spack/spack
$ spack install hdf5
```

## Simple syntax enables complex installs

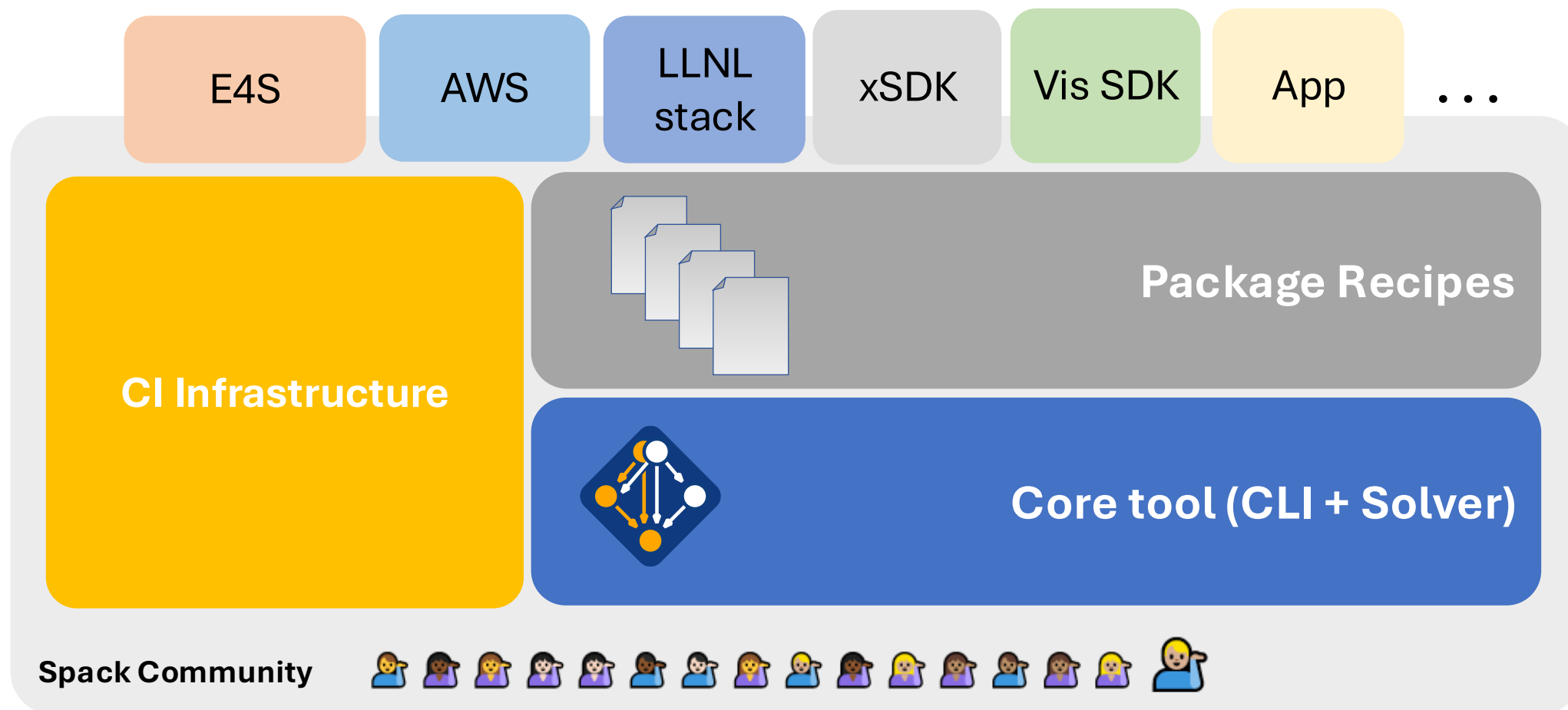
```
$ spack install hdf5@1.14
$ spack install hdf5@1.14 %clang@21
$ spack install hdf5@1.14 +threadssafe
$ spack install hdf5@1.14 cppflags="-O3 -g3"
$ spack install hdf5@1.14 target=haswell
$ spack install hdf5@1.14 +mpi ^mpich@5
```

- Generates, but does **not** require, *environment modules*
- Allows virtualenv-like *environments*



[github.com/spack/spack](https://github.com/spack/spack)

# What does the Spack project look like now?

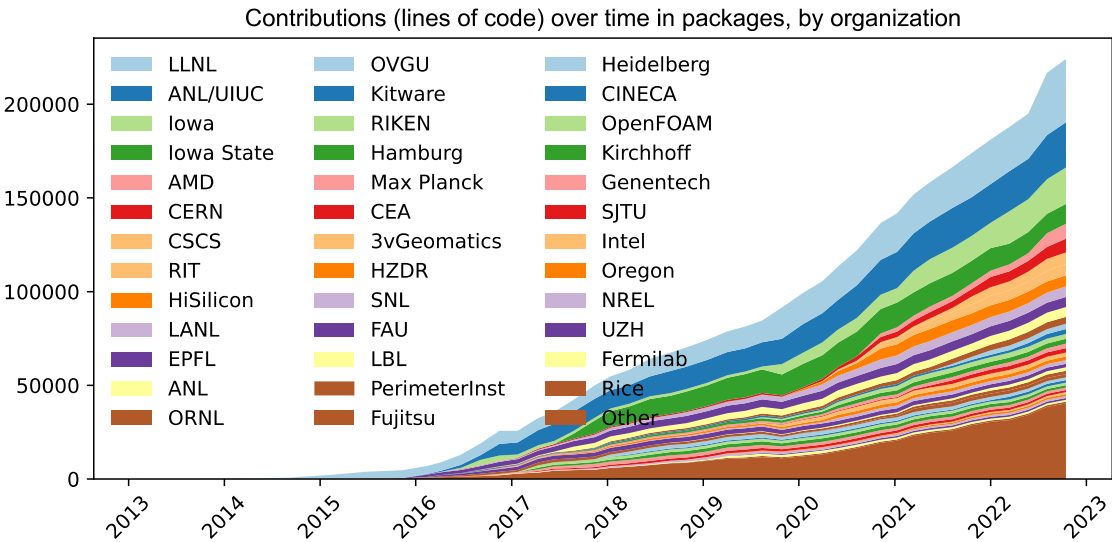


# Spack sustains the HPC software ecosystem with the help of many contributors



2023 aggregate documentation user counts from GA4  
(note: yearly user counts are almost certainly too large)

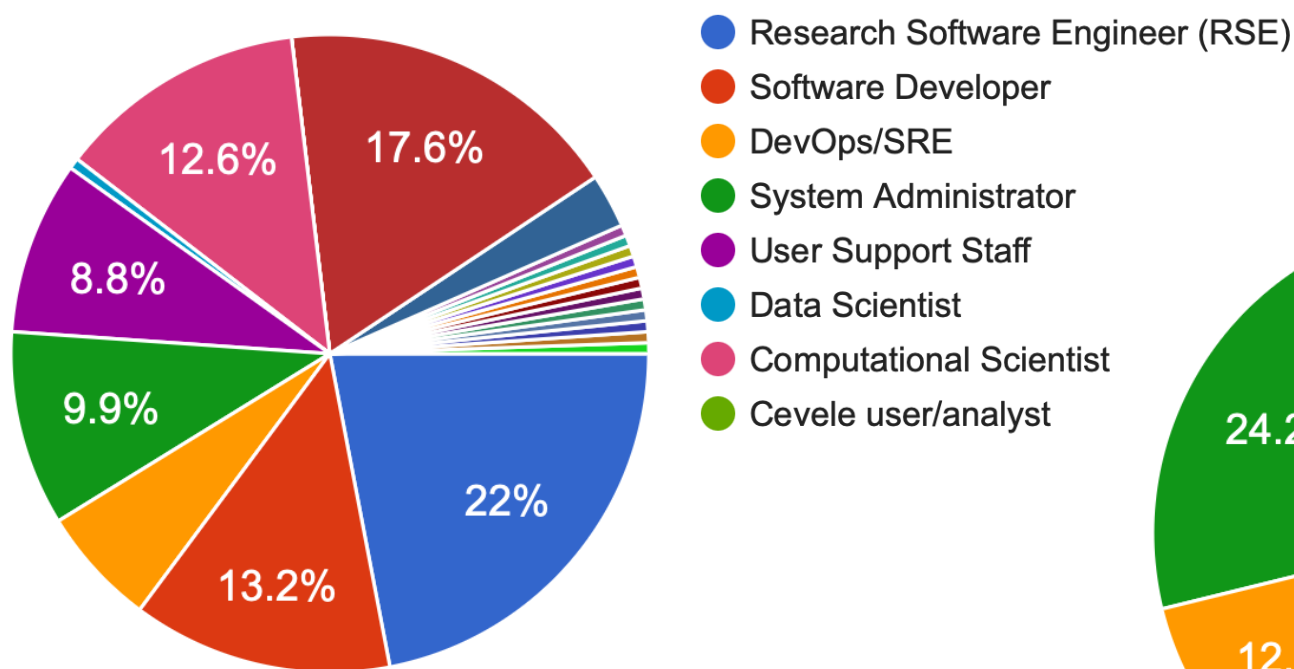
**Over 7,900** software packages  
**Over 1,300** contributors



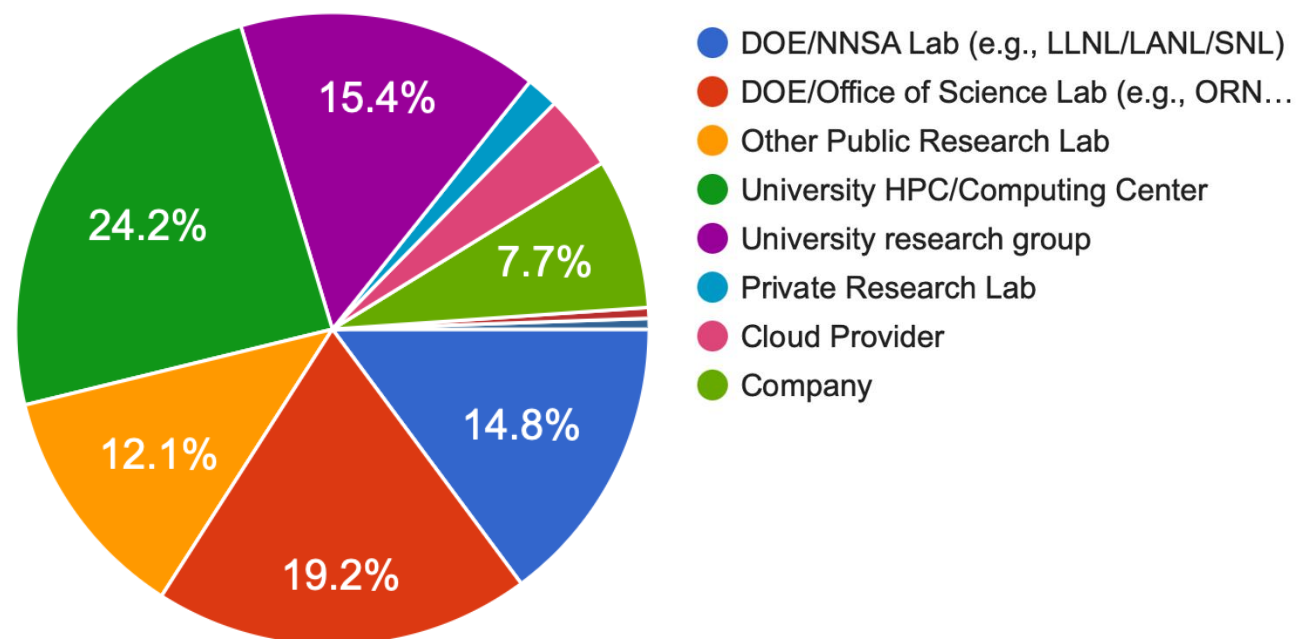
**Contributors continue to grow worldwide!**

# Spack User Base according to the last poll

## What type of user are you?



## Where do you work?



# Spack is a Linux Foundation Project!

## What does that mean?

- Spack has a legal entity
  - This is a neutral 501(c)(6) non-profit company
  - Can be in legal agreements (e.g. for distributing binaries)
- Spack has a Technical Steering Committee (TSC)
  - TSC meetings are public, anyone can attend
  - See GOVERNANCE.md in the Spack project
- *Trademark* assigned to Linux Foundation
- Spack project resources owned by Linux Foundation



Why are we doing this?

# Where can you reach the community?



★ Star us on GitHub!  
[github.com/spack/spack](https://github.com/spack/spack)



[slack.spack.io](https://slack.spack.io)



[@spack@hpc.social](https://matrix.to/#/!spack:hpc.social)



[@spackpm.bsky.social](https://bsky.app/profile/spackpm)



[@spackpm](https://x.com/spackpm)

# Spack basic concepts

# Spec syntax: describe package configurations

```
$ spack install mpileaks                unconstrained
$ spack install mpileaks@3.3            @ custom version
$ spack install mpileaks@3.3 %gcc@15.   % custom compiler
$ spack install mpileaks@3.3 %gcc@15. +threads +/- build option
$ spack install mpileaks@3.3 cppflags="-O3 -g3" set compiler flags
$ spack install mpileaks@3.3 target=cascadelake set target microarchitecture
$ spack install mpileaks@3.3 ^mpich@3.2 %gcc@15. ^ dependency constraints
```

- Each expression is a **spec** for a particular configuration
  - Each clause adds a constraint to the spec
  - Constraints are optional – specify only what you need.
  - Customize install on the command line!
- Spec syntax is recursive
  - Full control over the combinatorial build space

# Package recipes are *parameterized*

```
from spack import *

class Kripke(CMakePackage):
    """Kripke is a simple, scalable, 3D Sn deterministic particle transport mini-app."""

    homepage = "https://computation.llnl.gov/projects/co-design/kripke"
    url      = "https://computation.llnl.gov/projects/co-design/download/kripke-openmp-1.1.tar.gz"

    version('1.2.3', sha256='3f7f2eef0d1ba5825780d626741eb0b3f026a096048d7ec4794d2a7dfbe2b8a6')
    version('1.2.2', sha256='eaf9ddf562416974157b34d00c3a1c880fc5296fce2aa2efa039a86e0976f3a3')
    version('1.1', sha256='232d74072fc7b848fa2adc8a1bc839ae8fb5f96d50224186601f55554a25f64a')

    variant('mpi', default=True, description='Build with MPI.')
    variant('openmp', default=True, description='Build with OpenMP enabled.')

    depends_on('mpi', when='+mpi')
    depends_on('cmake@3.0:', type='build')

    def cmake_args(self):
        return [
            '-DENABLE_OPENMP=%s' % ('+openmp' in self.spec),
            '-DENABLE_MPI=%s' % ('+mpi' in self.spec),
        ]

    def install(self, spec, prefix):
        mkdirp(prefix.bin)
        install('./spack-build/kripke', prefix.bin)
```

**Base package**

(CMake support)

**Metadata** at the class level

**Versions**

**Variants** (build options)

**Dependencies**

(same spec syntax)

**Install logic**

in instance methods

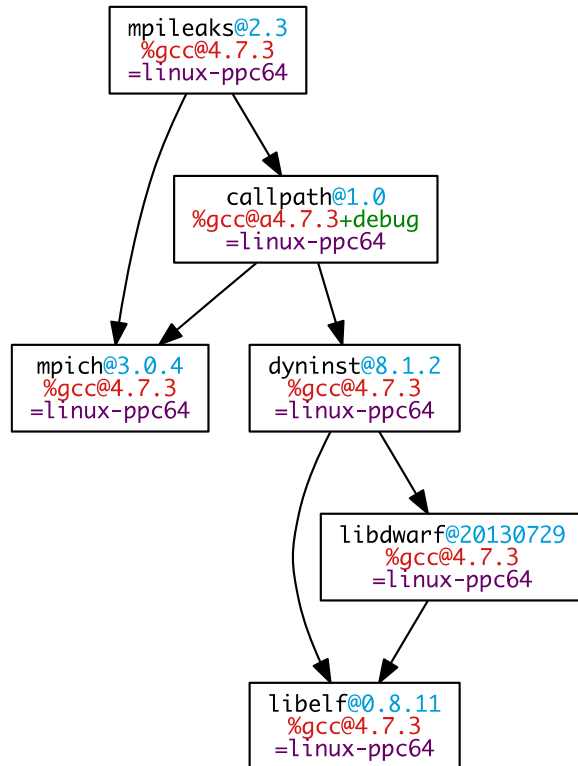
**One package.py file per software project!**

# Concretization fills in missing details

mpileaks ^callpath@1.0+debug ^libelf@0.8.11

User input: *abstract* spec with some constraints

Concretize



Concrete spec is fully constrained  
and can be passed to install.

Store

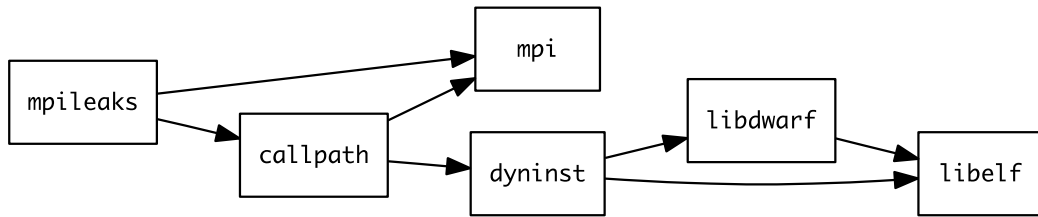
spec.json

```
spec:
- mpileaks:
  arch: linux-x86_64
  compiler:
    name: gcc
    version: 4.9.2
  dependencies:
    adept-utils: kszrtkpbzac3ss2ixcjkorlaybnptp4
    callpath: ba h5f4h4d2n47mgycej2mtrnrivvxy77
    mpich: aa4a6ifj23yijqm dabeakpejcli72t3
    hash: 33hjjxi7p6gyzn5ptgyes7sghyprujh
    variants: {}
    version: '1.0'
- adept-utils:
  arch: linux-x86_64
  compiler:
    name: gcc
    version: 4.9.2
  dependencies:
    boost: teesjv7ehpe5ksspjim5dk43a7qnowlq
    mpich: aa4a6ifj23yijqm dabeakpejcli72t3
    hash: kszrtkpbzac3ss2ixcjkorlaybnptp4
    variants: {}
    version: 1.0.1
- boost:
  arch: linux-x86_64
  compiler:
    name: gcc
    version: 4.9.2
  dependencies: {}
  hash: teesjv7ehpe5ksspjim5dk43a7qnowlq
  variants: {}
  version: 1.59.0
...
```

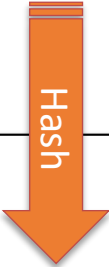
Detailed provenance is stored  
with the installed package

# Hashing allows us to handle combinatorial complexity

## Dependency DAG



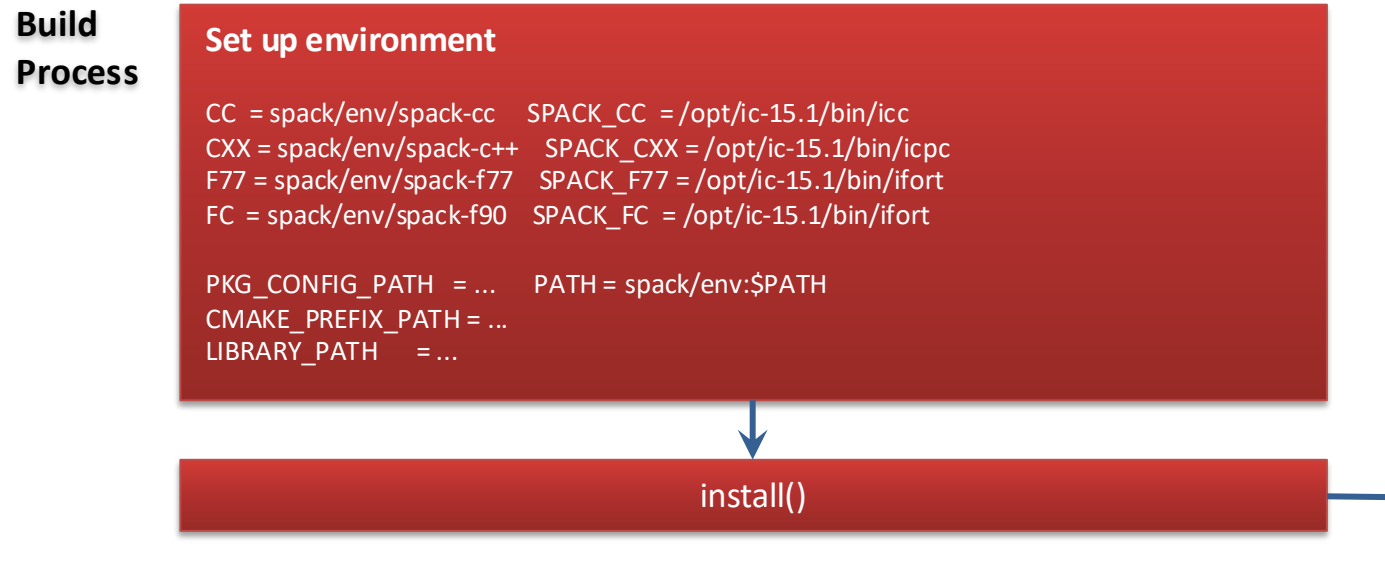
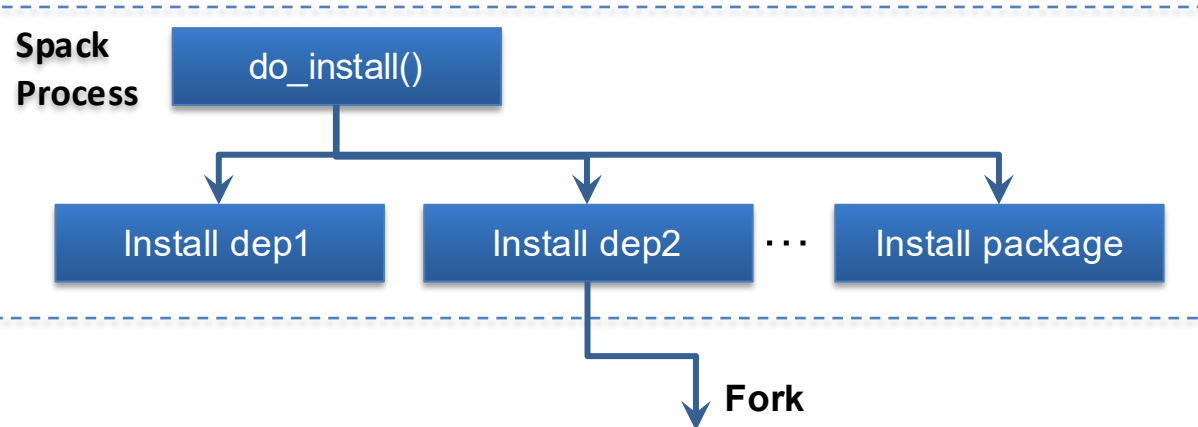
## Installation Layout



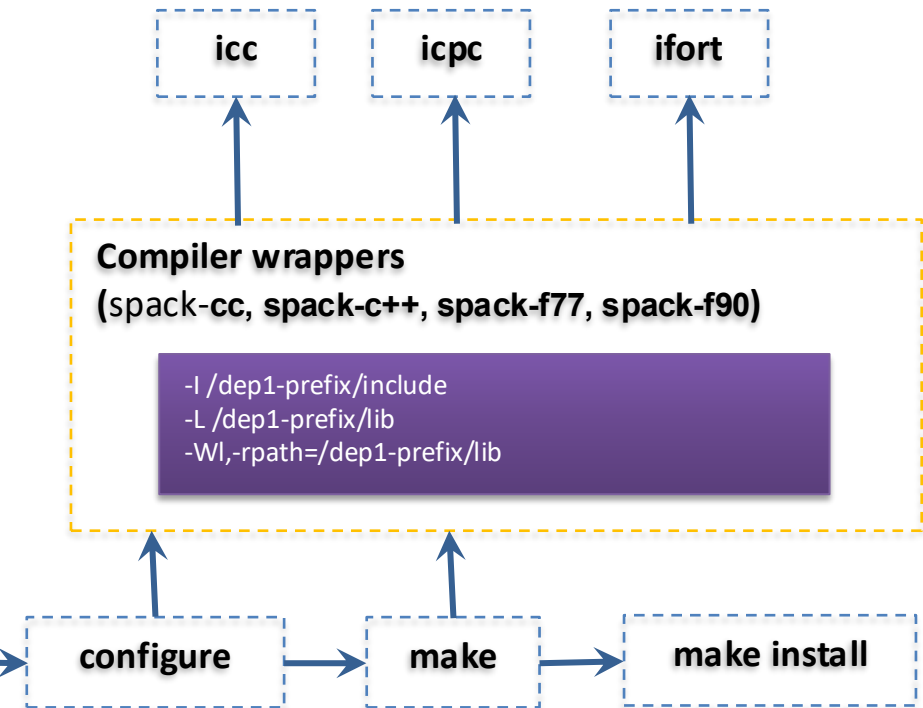
```
opt
└─ spack
   └─ darwin-mojave-skylake
      └─ clang-10.0.0-apple
         ├── bzip2-1.0.8-hc4sm4vuzpm4znmvrfzri4ow2mkphe2e
         ├── python-3.7.6-daqqpsxb6qbfrztsezkmhus3xoflby
         ├── sqlite-3.30.1-u64v2igxvyn23hysmklfums6tgjv5r
         ├── xz-5.2.4-u5eawkvaoc7vonabe6nndkcfwuv233cj
         └─ zlib-1.2.11-x46q4wm46ay4pltrijbgizxjrhbaka6
      └─ darwin-mojave-x86_64
         └─ clang-10.0.0-apple
            └─ coreutils-8.29-pl2kcytejqcys5dzecfrtjqxfsdssvnb
```

- Each unique dependency graph is a unique **configuration**.
- Each configuration in a unique directory.
  - Multiple configurations of the same package can coexist.
- **Hash** of entire directed acyclic graph (DAG) is appended to each prefix.
- Installed packages automatically find dependencies
  - Spack embeds RPATHs in binaries.
  - No need to use modules or set LD\_LIBRARY\_PATH
  - Things work *the way you built them*

# An isolated compilation environment allows Spack to easily swap compilers



- **Forked build process isolates environment for each build.**  
**Uses compiler wrappers to:**
  - Add include, lib, and RPATH flags
  - Ensure that dependencies are found automatically
  - Load Cray modules (use right compiler/system deps)



# Spack environments close the circle with views



## spack.yaml

```
spack:
  # include external configuration
  include:
  - ../special-config-directory/
  - ./config-file.yaml

  # add package specs to the `specs` list
  specs:
  - hdf5
  - libelf
  - openmpi
```

## spack.lock

```
{
  "concrete_specs": {
    "6s63so2kstp3zyvjezgldmavy6l3nul": {
      "hdf5": {
        "version": "1.10.5",
        "arch": {
          "platform": "darwin",
          "platform_os": "mojave",
          "target": "x86_64"
        },
        "compiler": {
          "name": "clang",
          "version": "10.0.0-apple"
        },
        "namespace": "builtin",
        "parameters": {
          "cxx": false,
          "debug": false,
          "fortran": false,
          "hl": false,
          "mpi": true,

```

# Let's move on to demos...



# Extra slides on Spack v1.x

## **EB + EESSI + Spack Hackathon**

CINECA (BO), May 19-21, 2026

---

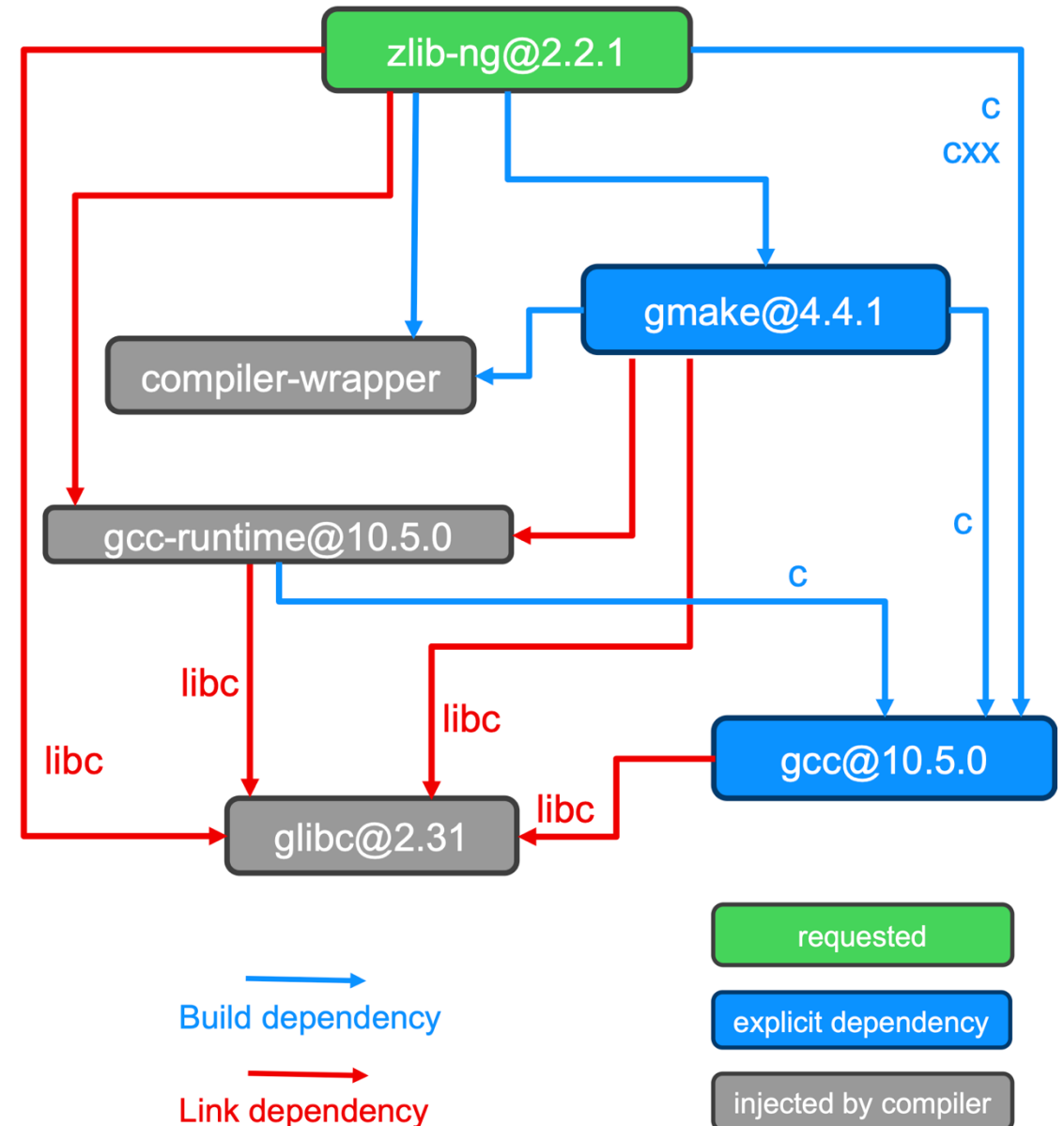
Harmen Stoppels, Massimiliano Culpo

**Spack v1.0**

# Compiler dependencies

# Compiler Dependencies

- Compilers are now build dependencies
  - % changes semantics in specs
- Runtime libraries modeled as packages
  - gcc-runtime is injected as link dependency by gcc
  - packages depend on c, cxx, fortran virtuals, which are satisfied by gcc node
- glibc is an automatically detected external
  - Injected as a `libc` virtual dependency
  - Does not require user configuration
- Will eventually be able to choose implementations (e.g., musl)
- No *compilers.yaml* anymore



# Language dependencies in package.py

```
depends_on("c", type="build")  
depends_on("cxx", type="build")  
depends_on("fortran", type="build")
```

- Spack has historically made these compilers available to every package
  - A compiler was actually “something that supports c + cxx + fortran + f77”
  - Made for a lot of special cases
  - Also makes for duplication of purely interpreted packages (e.g. python)
- Required in 1.0 if you want to use c, cxx, or fortran
  - No-op in v0.23 and prior as we prepared for this feature

# Spack v0.23: % ordering and forward compatibility

You could previously write:

```
pkg %gcc +foo    # +foo would associate with pkg, not gcc – will error in 1.0
```

Now you'll need to write:

```
pkg +foo %gcc    # +foo associates with pkg
```

We want these to be symmetric:

```
pkg +foo %dep +bar # `pkg +foo` depends on `dep +bar` directly  
pkg +foo ^dep +bar # `pkg +foo` depends on `dep +bar` directly or transitively
```

To change your files in a forward compatible way automatically:

```
$ spack style --spec-strings --fix
```

# Compiler dependencies and changes in semantics

## Spack v0.23

```
# Is Intel the C compiler?
if self.compiler.cc == "icc":
    ...

# Get CMake from a transitive dependency
cmake = self["cmake"].command

# Check if our direct gmake dependency
# is above 4.4
if self.spec.satisfies("^gmake@4.4:"):
    ...

# Grab the C++ compiler
cxx = self.compiler.cxx
```

## Spack v1.0

```
# Is Intel the C compiler?
if self.spec.satisfies("%c=oneapi"):
    ...

# Get CMake from a transitive dependency
cmake = self[" hdf5 "]["cmake"].command

# Check if our direct gmake dependency
# is above 4.4
if self.spec.satisfies("%gmake@4.4:"):
    ...

# Grab the C++ compiler
cxx = self[" cxx "].cxx
```

# Spack v1.0 introduces *toolchains*

```
toolchains:  
  clang_gfortran:  
    - spec: %c=clang  
      when: %c  
    - spec: %cxx=clang  
      when: %cxx  
    - spec: %fortran=gcc  
      when: %fortran  
    - spec: fflags="-ffree-line-length-none"
```

```
toolchains:  
  oneapi_mvapich2:  
    - spec: %c=oneapi@2025.1.1  
      when: %c  
    - spec: %cxx=oneapi@2025.1.1  
      when: %cxx  
    - spec: %fortran=oneapi@2025.1.1  
      when: %fortran  
    - spec: mvapich2 @2.3.7  
      when: %mpi
```

```
$ spack install gromacs %clang_gfortran  
$ spack install gromacs %oneapi_mvapich2
```

**Spack v1.0**

# Splitting the repository

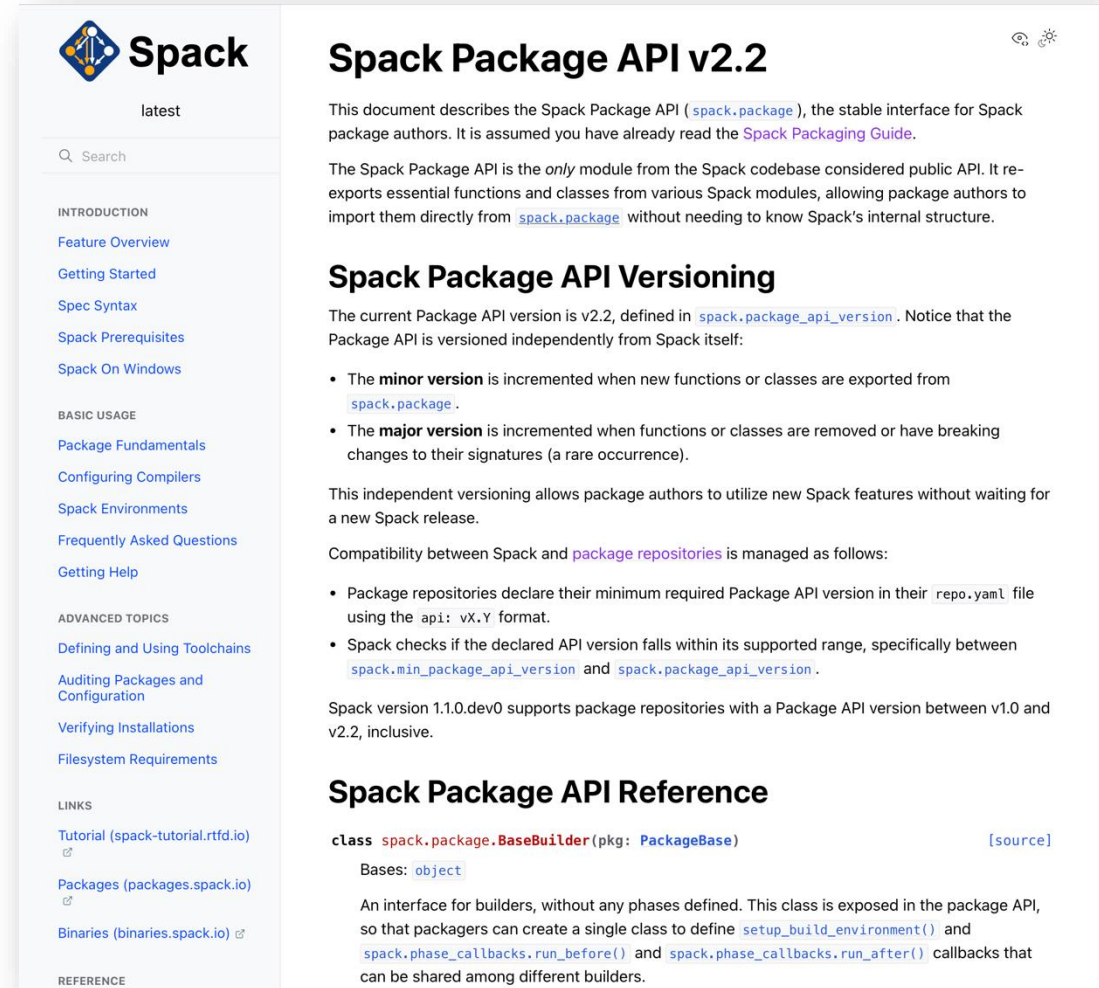
# Package repository: why splitting?

- Spack is two things:
  - Command line tool `spack`
  - Package repository with 8,600+ recipes
- Community wanted
  - package updates without tool changes (e.g. new bugs)
  - tool updates without package changes (reproducibility)
- But coupling between tool and packages was tight
  1. Package classes are in core: `CMakePackage`, `AutotoolsBuilder`, etc.
  2. Compiler wrapper was not a package until recently
  3. Packages *live* in Spack's GitHub repository with a long (git) history

# Spack now has a Stable Package API

- Repositories define API version used
  - Versioned *per commit*
- Spack defines API version(s) supported
  - Bumping minimum supported version is a major breaking change
  - Spack complains if a repo is too new
- Packages can only import from:
  - `spack.package`
  - Core Python

[https://spack.rtf.d.io/en/latest/package\\_api.html](https://spack.rtf.d.io/en/latest/package_api.html)



The screenshot shows the Spack Package API v2.2 documentation page. The left sidebar contains a navigation menu with sections: INTRODUCTION (Feature Overview, Getting Started, Spec Syntax, Spack Prerequisites, Spack On Windows), BASIC USAGE (Package Fundamentals, Configuring Compilers, Spack Environments, Frequently Asked Questions, Getting Help), ADVANCED TOPICS (Defining and Using Toolchains, Auditing Packages and Configuration, Verifying Installations, Filesystem Requirements), LINKS (Tutorial, Packages, Binaries), and REFERENCE. The main content area is titled "Spack Package API v2.2" and includes an introduction, versioning details, compatibility information, and a reference section for the `BaseBuilder` class.

## Spack Package API v2.2

This document describes the Spack Package API (`spack.package`), the stable interface for Spack package authors. It is assumed you have already read the [Spack Packaging Guide](#).

The Spack Package API is the *only* module from the Spack codebase considered public API. It re-exports essential functions and classes from various Spack modules, allowing package authors to import them directly from `spack.package` without needing to know Spack's internal structure.

### Spack Package API Versioning

The current Package API version is v2.2, defined in `spack.package_api_version`. Notice that the Package API is versioned independently from Spack itself:

- The **minor version** is incremented when new functions or classes are exported from `spack.package`.
- The **major version** is incremented when functions or classes are removed or have breaking changes to their signatures (a rare occurrence).

This independent versioning allows package authors to utilize new Spack features without waiting for a new Spack release.

Compatibility between Spack and [package repositories](#) is managed as follows:

- Package repositories declare their minimum required Package API version in their `repo.yaml` file using the `api: vx.y` format.
- Spack checks if the declared API version falls within its supported range, specifically between `spack.min_package_api_version` and `spack.package_api_version`.

Spack version 1.1.0.dev0 supports package repositories with a Package API version between v1.0 and v2.2, inclusive.

### Spack Package API Reference

```
class spack.package.BaseBuilder(pkg: PackageBase) [source]
    Bases: object
```

An interface for builders, without any phases defined. This class is exposed in the package API, so that packagers can create a single class to define `setup_build_environment()` and `spack.phase_callbacks.run_before()` and `spack.phase_callbacks.run_after()` callbacks that can be shared among different builders.

**Spack v1.2**

# Concretizer performance

# Recent performance improvements

- clingo binaries with PGO, LTO, mimalloc (old news)
- Benchmarking and performance regression tests
- ground + solve
  - “dimension reduction” in problem encoding
- setup
  - Lazy package metadata evaluation
  - Immutable abstract specs and reduced allocations

# Concretizer performance across Spack versions

